

Neuronauts v2: Scaffolded Global Grammar for End-to-End Connectome Inference

Neuronauts

March 2025

Contents

1	Neuronauts v2: Scaffolded Global Grammar for End-to-End Connectome Inference	2
1.1	Plain Language Summary	2
1.2	Abstract	3
1.3	1. Introduction	3
1.4	2. System Architecture	3
1.4.1	2.1 Perception Layer	4
1.4.2	2.2 Scaffold-Aware Initialisation	5
1.4.3	2.3 Shared Grammar Layer	5
1.4.4	2.4 Bridge Proposals	6
1.4.5	2.5 Global Assembly GAT	6
1.5	3. Real-Data Training Infrastructure	6
1.5.1	3.1 Box Cache and Selection	6
1.5.2	3.2 End-to-End Training CLI	7
1.6	4. Supervision and Loss Functions	7
1.6.1	4.1 Local Merge Loss	7
1.6.2	4.2 Atomicity Loss	7
1.6.3	4.3 Bridge Loss (Self-Supervised)	7
1.6.4	4.4 GAT Soft-F1 Loss	8
1.7	5. Evaluation	8
1.8	6. Implementation Status	8
1.9	7. Methods Summary	9
1.9.1	7.1 Scaffold Union-Find	9
1.9.2	7.2 Merge Scoring and Beam Search	9
1.9.3	7.3 Line-Graph F1 Definition	9
1.9.4	7.4 Design Choices and Rationale	9
1.10	8. Related Work and Positioning	10
1.10.1	8.1 Relationship to Infrastructure and Data	10
1.10.2	8.2 Relationship to Segmentation and Tracing	10
1.10.3	8.3 Relationship to Proofreading and Edit Imitation	11
1.10.4	8.4 Detailed Comparison: NEURD, Guided Proofreading, and Neuronauts	11
1.10.5	8.5 What Neuronauts Is Not	13
1.11	9. Conclusion	14
1.12	References	14

1 Neuronauts v2: Scaffolded Global Grammar for End-to-End Connectome Inference

1.1 Plain Language Summary

The problem. Neuroscientists want to know which neurons are connected to which. To figure this out they use electron microscopy (EM) — images of brain slices so detailed you can see individual cell membranes. The challenge is that a single cubic millimeter of brain contains millions of neural processes (axons, dendrites) that must all be traced and connected correctly. Doing this by hand takes years; automated tools are still error-prone.

What we have to work with. The MICrONS project at the Allen Institute has already imaged a cubic millimeter of mouse visual cortex at nanometer resolution and assigned preliminary IDs (called “root IDs”) to each neuron using a tool called CAVE. These IDs are imperfect — neurons are sometimes split across multiple IDs, or two neurons are incorrectly merged into one — but they encode real spatial information. Every synapse (connection point) has a known location and is tagged with the root IDs of the pre- and post-synaptic neurons.

What Neuronauts does. Rather than starting from scratch, Neuronauts treats the CAVE IDs as a noisy scaffold and learns to correct them. The system asks: “given what I can see about how neural processes move through this little cube of tissue, which synapses belong to the same neuron — and therefore which neurons are actually connected?”

How it works, step by step:

1. **Fetch a small cube of EM data** ($\sim 6\ \mu\text{m}$ per side) from MICrONS and pull all the synapse locations and root IDs for that cube from CAVE.
2. **Run 700 virtual agents** through the EM cube. Each agent starts near a synapse and walks through the tissue, repelled by membranes (cell walls), attracted toward synapses, and driven by an exploration field. After 450 steps each agent has traced a path that represents its best guess at following a single neural process. The agents record which synapses they visit along the way.
3. **Use CAVE IDs as a head start.** Before doing anything learned, agents that visited synapses from the same CAVE supervoxel are pre-grouped. This cheap step reduces the problem by $\sim 10\times$ — instead of asking “which of 700 agents belong together?” we ask “which of ~ 70 groups belong together?”
4. **Score pairwise compatibility.** A small Transformer neural network (the “grammar”) reads each agent’s path — encoded as a sequence of (step length, distance from center, turning angle) values — and produces a compact embedding. Two paths are likely from the same neuron if their embeddings are similar. The grammar is trained to distinguish same-neuron fragments from different-neuron fragments.
5. **Build a connectivity graph.** Based on the grammar scores and spatial proximity, fragments are merged into candidate neurons, each claimed neuron is assigned ownership of nearby synapses, and the result is a graph: neurons as nodes, synapses as edges.
6. **Refine with a global network (GAT).** A Graph Attention Network looks at the full connectivity graph, considers all neurons and edges simultaneously, and prunes or confirms connections based on global context — catching errors that purely local decisions miss.
7. **Evaluate against ground truth.** The quality of the result is measured by synapse line-graph F1: if two synapses share the same pre-synaptic neuron in our predicted graph, they should also share the

same CAVE root ID in reality (and vice versa). This F1 score is the single number that all training targets.

What makes this different. Most connectomics tools optimize for intermediate goals like “how accurately did we trace this membrane segment?” Neuronauts optimizes directly for the thing you actually care about — the correctness of the induced connectome — using the final F1 score as the training signal at every level.

Current state. The full architecture is implemented and tested. We are now training on real MICrONS data for the first time, building a baseline F1 score against which all future improvements will be measured.

1.2 Abstract

Modern connectomics pipelines are fragmented across three loosely coupled stages: EM perception, local reconstruction or proofreading, and downstream connectome extraction. This decomposition creates an optimization gap — learned components are trained on proxy tasks (membrane segmentation, local merge AUC, edit imitation) while the scientific target is the connectome itself.

Neuronauts v2 closes this gap by framing connectomics as a graph-refinement problem over existing CAVE segmentations. Instead of building connectivity from scratch, we treat each CAVE segment as a noisy scaffold node and learn to merge, split, and bridge them using a multi-modal Transformer-GNN architecture trained end-to-end against synapse line-graph F1. The system contains four learned components — a Transformer path encoder, a trajectory bridge head, a scaffold-aware graph initializer, and a global Graph Attention Network — all sharing a common coordinate-free path representation. On the MICrONS Minnie65 dataset, scaffold initialization reduces the combinatorial search space approximately $10\times$ before any learned decision is made; the subsequent GlobalAssemblyGAT, trained with a differentiable soft-F1 surrogate, further improves edge precision over heuristic beam search baselines.

1.3 1. Introduction

The practical goal in connectomics is not acceptable voxel segmentation but object identity at the synapse level: which synapses belong to the same neuronal process and therefore which connections exist in the final connectome. Segmentation errors matter only insofar as they alter that induced topology.

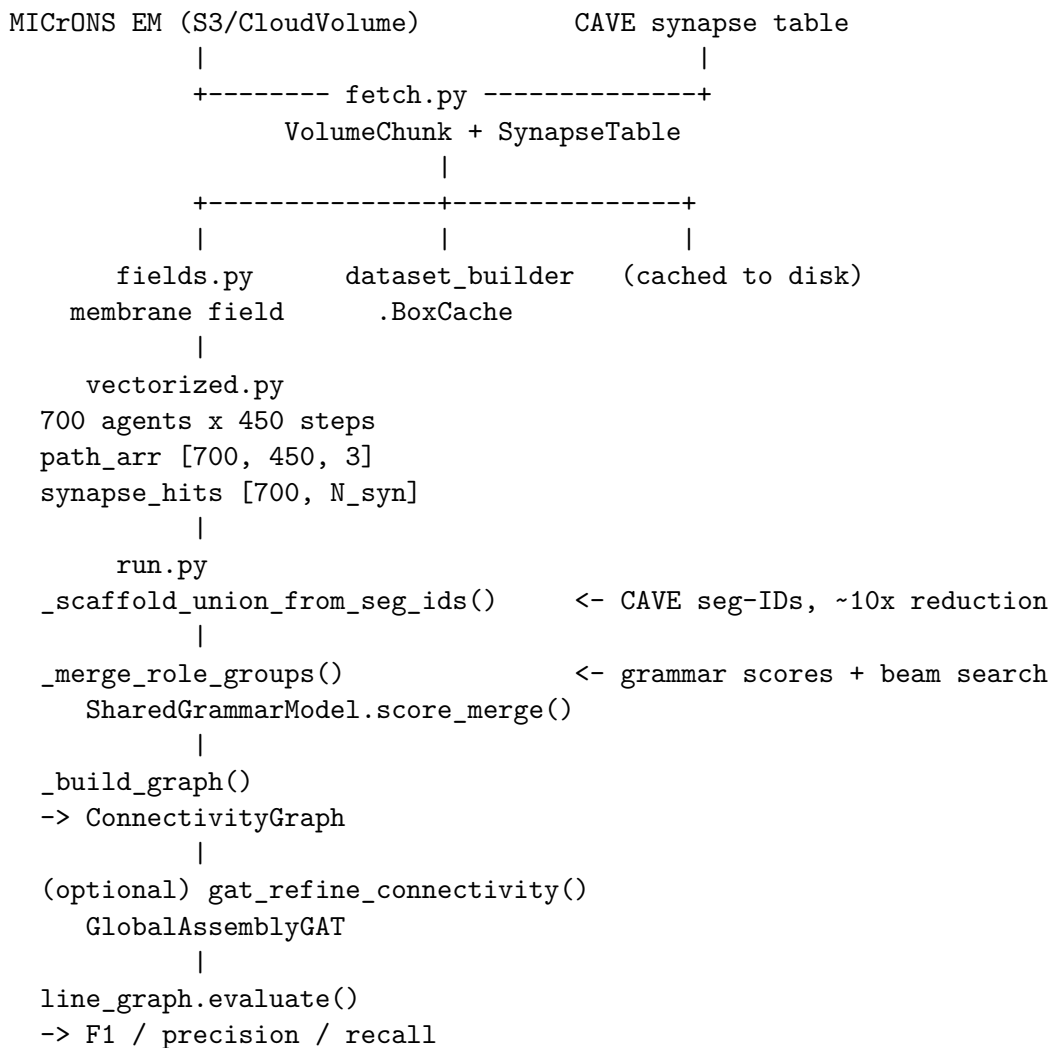
This motivates a different decomposition from the classical perception–reconstruction–evaluation pipeline:

1. Produce local fragment evidence from EM.
2. Use existing coarse segmentations (CAVE root IDs) as a noisy scaffold.
3. Learn a shared grammar for neurite structural compatibility.
4. Evaluate candidate assemblies by their effect on synapse line-graph correctness.

Consequence one: local metrics such as merge AUC remain useful as internal health signals but are not the primary training target. Consequence two: the terminal scalar should be line-graph F1 over synapses — the closest box-scale proxy for downstream connectome quality. Consequence three: the existing CAVE segmentation, despite its errors, encodes substantial spatial structure that can be exploited to collapse the search space before learned decisions are made.

1.4 2. System Architecture

The full data and control flow:



1.4.1 2.1 Perception Layer

Files: neuronauts/fetch.py, neuronauts/fields.py, neuronauts/vectorized.py, neuronauts/membrane_unet

The perception layer fetches MICrONS/CAVE boxes and synapses, computes local field guidance (membrane repulsion, exploration, synapse attraction), and runs a vectorized swarm of 700 agents for 450 steps per box. Each agent traces a path through the EM volume guided by the local fields and records which synapse sites it visits.

2.5D Membrane U-Net. Membrane prediction uses a U-Net that fuses a central Z-slice with ± 2 neighbouring slices as extra input channels (`context_slices=2` default). This provides 3D context at the cost of 2D convolutions and is calibrated for MICrONS's 8 nm in-plane / 40 nm axial anisotropy. Instance normalisation (`InstanceNorm2d`) replaces batch normalisation, enabling stable training at batch size 1 and stable inference at arbitrary spatial scales. Without a trained checkpoint, the system falls back to Sobel-gradient-based membrane estimation.

Feature extraction. Beyond raw grayscale, `fetch.py` exposes per-step skeleton features (step distance, normalised arc-length position, turning angle) and mesh features (volume-to-surface ratio, per-vertex projections) for downstream multi-modal encoding.

1.4.2 2.2 Scaffold-Aware Initialisation

Files: `neuronauts/run.py(_scaffold_union_from_seg_ids)`, `neuronauts/fetch.py(SynapseTable)`

Before any learned grammar decision is made, agents that belong purely to the same CAVE segment ID are pre-merged into scaffold groups. Each synapse’s `pre_pt_supervoxel_id` / `post_pt_supervoxel_id` serves as a noisy but informative prior: if two agents visit synapses from the same supervoxel, they almost certainly belong to the same true neuron.

`SynapseTable` carries `pre_seg_id` and `post_seg_id` fields populated from CAVE materialization tables. `_scaffold_union_from_seg_ids` applies union-find over these IDs before the main merge stage, collapsing the agent search space approximately $10\times$ — from $O(N^2)$ pairwise merge decisions to $O(K^2)$ where K is the number of scaffold groups.

HeuristicConfig. All spatial thresholds (`MERGE_RADIUS`, `MERGE_OVERLAP_THRESHOLD`, `POLARITY_CAPTURE_R`, `MAX_SYNAPSES_PER_NEURON`) are encapsulated in a frozen `HeuristicConfig` dataclass. When any trained component (grammar, GAT) is present, the runtime switches to `HeuristicConfig.learned()` — spatial thresholds become candidate generators rather than hard decisions, and the learned components make the final accept/reject calls.

1.4.3 2.3 Shared Grammar Layer

Files: `neuronauts/grammar.py`, `neuronauts/shared_grammar_model.py`

Coordinate system. All path features are computed in isotropic 32-nm units before being fed to the encoder. MIP-2 voxel coordinates ($32 \times 32 \times 40$ nm/vox) are rescaled by $[1.0, 1.0, 1.25]$ so that Z-axis steps are correctly weighted relative to XY ($40/32 = 1.25\times$) while keeping feature magnitudes in the numerically stable range of ~ 1 –60 units. The curvature feature uses normalised direction vectors and is scale-independent. Without this correction, the 25% Z anisotropy would distort learned path geometry — a diagonal path through Z would appear artificially longer than the equivalent XY path.

Transformer PathEncoder. The core path representation replaces heuristic pooling with a `nn.TransformerEncoder` stack. Input features (`edge_len`, `radius`, `curvature`) — plus optional per-step skeleton and mesh descriptors — are projected to a `d_model`-dimensional embedding and concatenated with sinusoidal positional encodings. A learned [CLS] token is prepended; its output after the Transformer stack is the global fragment representation. The encoder is coordinate-free (independent of absolute position) and reusable across volumes. `enable_nested_tensor=False` is set on the `TransformerEncoder` to opt out of the experimental PyTorch nested-tensor fast path, which is not needed given the explicit [CLS] masking.

Multitask training. `SharedGrammarModel` is trained jointly on three tasks via `multitask_train_step`, which returns `merge_accuracy`, `atomicity_accuracy`, and all loss scalars:

- *Local merge:* binary classification — should two path fragments be merged? Positives are subfragments from the same CAVE root; negatives are nearby fragments from different roots.
- *Global atomicity:* binary classification — is a synapse-side cluster atomic (one true root) or non-atomic (mixed)?
- *Self-supervised bridge:* MSE + cosine loss on predicted midpoint and tangent direction between fragment endpoints. No manual labels — geometric targets come from synapse positions.

BridgeHead. A 3-layer MLP takes `[left_emb, right_emb, left_emb - right_emb]` and predicts a 6D vector: 3D midpoint position + 3D tangent direction.

1.4.4 2.4 Bridge Proposals

Files: `neuronauts/dijkstra.py`, `neuronauts/run.py` (`_build_bridge_graph`, `_propose_bridges`)

In regions where the EM is corrupted or agent simulation fails to establish connections, the system falls back to Dijkstra-based bridge proposals. A `BridgeGraph` is constructed over fragment endpoints with edge costs derived from Euclidean proximity and bridge-head prediction confidence. These proposals provide a pool of candidate connections that the GAT can subsequently score.

1.4.5 2.5 Global Assembly GAT

Files: `neuronauts/assembly.py`, `neuronauts/shared_grammar_model.py`

Architecture. `GlobalAssemblyGAT` is a 2-layer stacked sparse Graph Attention Network. Each `_SparseGATLayer` computes multi-head attention using scatter operations over the sparse edge list — no dense adjacency matrix. Node features are path-encoder embeddings of each scaffold neuron; edges come from synapse connectivity. The output is globally-contextualised node embeddings from which `score_edges` produces per-synapse logits.

Training. The GAT training loop is grounded directly in the terminal metric:

1. `label_graph_edges` assigns binary labels to each edge in a `ConnectivityGraph` using majority-vote root-ID matching — the direct per-edge analogue of line-graph F1.
2. `gat_train_step` minimises $(1 - w) \times \text{BCE} + w \times (1 - \text{soft_F1})$ where $\text{soft_F1} = 2\text{TP} / (2\text{TP} + \text{FP} + \text{FN} + \epsilon)$ is differentiable through sigmoid probabilities. The path encoder is frozen during GAT training.
3. Agent simulation is required (~20–60 s/box on CPU); use `--gat-every-n-epochs 5` to amortise this cost.

Inference. `gat_refine_connectivity` encodes all neurons, runs GAT message passing, scores each synapse edge, and returns a refined graph containing only edges above the configured threshold.

1.5 3. Real-Data Training Infrastructure

1.5.1 3.1 Box Cache and Selection

File: `neuronauts/dataset_builder.py`

Each training box is cached as `<hash>.npz` (volume + synapse arrays) and `<hash>.json` (metadata), with an `index.json` manifest. Features are computed on-the-fly from the cached coordinates at training time, so changes to the feature extraction pipeline do not require re-fetching boxes.

Three box selection strategies are available:

- **Synapse-seeded sampling** (`select_synapse_seeded_boxes`, default): queries CAVE for up to 2000 real synapse positions and randomly samples N of them as box centres. Every box is guaranteed to land in the annotated neuropil. This is the only reliable strategy when nucleus table files are unavailable — uniform random sampling over the declared Minnie65 extents (3.5 mm × 2.4 mm) mostly hits unannotated tissue.
- **Soma-centred selection** (`select_boxes_from_nucleus_table`): uses a static synapse-count TSV and nucleus detection CSV to bias sampling toward neurons with specific synapse-count ranges.
- **Random spatial sampling** (`select_random_boxes`): uniform sampling inside declared bounds. Produces many empty boxes; use only with `--n-boxes` set much higher than the target cache size.

`build_dataset` fetches each spec from CloudVolume and CAVE, filters by synapse count, and caches to disk. Already-cached boxes are skipped. The cache is not thread-safe; build datasets sequentially.

1.5.2 3.2 End-to-End Training CLI

File: `scripts/train.py`

Three subcommands:

- `build-dataset`: fetch and cache real MICrONS boxes. Defaults to synapse-seeded selection.
- `train`: grammar training from cached tables (~0.2–0.5 s/box/epoch on CPU, no simulation) plus optional GAT training (~30 s/box when a GAT epoch fires). Checkpoints saved whenever validation F1 improves. Validation uses the live in-memory model weights each epoch — not a cached checkpoint — so the logged `val_f1` tracks training progress in real time.
- `run`: build-dataset then train in one shot.

Quick start on a laptop (CPU):

```
# Fetch 40 boxes (synapse-seeded, guaranteed non-empty)
python scripts/train.py build-dataset \
  --cache-dir data/boxes \
  --n-boxes 40

# Train grammar only (skip --train-gat; GAT requires ~30s/box/step)
python scripts/train.py train \
  --cache-dir data/boxes \
  --epochs 30 \
  --grammar-output models/shared_grammar_real.pt
```

1.6 4. Supervision and Loss Functions

1.6.1 4.1 Local Merge Loss

Binary cross-entropy over pairs of path-sequence embeddings. Positives: subfragments from the same CAVE root cluster (split at the median PCA projection). Negatives: pairs of nearby clusters with different root IDs, sorted by centroid distance and capped at `max_negative_pairs_per_role`. Source: cached synapse tables — no agent simulation required.

1.6.2 4.2 Atomicity Loss

Binary cross-entropy over multi-branch cluster embeddings. Positive: a cluster whose synapses all share one root on the relevant side. Negative: a cluster formed by merging two distinct roots. Source: cached synapse tables — no agent simulation required.

1.6.3 4.3 Bridge Loss (Self-Supervised)

$\text{MSE}(\text{predicted_midpoint}, \text{true_midpoint}) + (1 - \text{cosine_similarity}(\text{predicted_direction}, \text{true_direction}))$. Targets are computed from geometric properties of adjacent synapse clusters — no manual annotation. Weight: `bridge_loss_weight = 0.5`.

1.6.4 4.4 GAT Soft-F1 Loss

$(1 - w) \times \text{BCE}(\text{logits}, \text{labels}) + w \times (1 - 2\text{TP} / (2\text{TP} + \text{FP} + \text{FN} + e))$ where $w = 0.5$. Soft-F1 operates on sigmoid probabilities so gradients flow through TP, FP, FN counts. This directly aligns GAT optimisation with the terminal line-graph metric. Requires agent simulation.

1.7 5. Evaluation

Shift to line-graph evaluation. Most connectomics pipelines evaluate at the segmentation level (merge AUC, per-neuron synapse precision/recall) or at the trace level (path accuracy). Neuronauts shifts evaluation to the **synapse line graph**: synapses are nodes, and an edge connects two synapses iff they share the same pre- or post-synaptic neuron. This formulation directly targets connectome topology — which synapses belong together — rather than proxy metrics. NEURD and similar tools report synapse precision/recall over manually proofread neurons; we optimise for line-graph F1 over synapse pairs, the closest box-scale proxy for downstream connectivity correctness.

The primary evaluation metric is **synapse line-graph F1**:

```
True graph:  edge (i, j) iff pre_root_id[i] == pre_root_id[j]
              or post_root_id[i] == post_root_id[j]
Predicted:   edge (i, j) iff synapses i and j are assigned to the
              same pre- or post-neuron in the ConnectivityGraph.
```

$$\text{F1} = 2 \cdot |\text{TP}| / (2 \cdot |\text{TP}| + |\text{FP}| + |\text{FN}|)$$

Supporting diagnostics logged alongside F1: precision, recall, merge accuracy, atomicity accuracy, and (when applicable) GAT edge precision/recall and hypothesis reranker correlation.

A leaderboard (`run_logs/research_ledger.leaderboard.tsv`) tracks all experiments ranked by hold-out F1.

1.8 6. Implementation Status

All five planned architectural components are implemented and tested (329+ unit/integration tests, with no warnings):

Component	File(s)	Tests
Transformer PathEncoder + [CLS]	<code>grammar.py</code>	<code>test_grammar_gaps.py</code> , <code>test_shared_grammar_training.py</code>
Isotropic coordinate scaling ($Z=1.25\times$)	<code>run.py</code> , <code>merge_dataset.py</code> , <code>topology_dataset.py</code> , <code>assembly.py</code>	<code>test_gat_assembly.py::test_z_step_is_10</code>
BridgeHead + Dijkstra proposals	<code>dijkstra.py</code> , <code>shared_grammar_model.py</code> , <code>run.py</code>	<code>test_bridge.py</code>
Scaffold init (<code>_scaffold_union_from_seg_ids</code>)	<code>run.py</code> , <code>fetch.py</code>	<code>test_scaffold.py</code>
2.5D MembraneUNet	<code>membrane_unet.py</code>	<code>test_membrane_unet.py</code>
GlobalAssemblyGAT + training	<code>assembly.py</code> , <code>shared_grammar_model.py</code>	<code>test_gat_assembly.py</code> , <code>test_gat_training.py</code>

Component	File(s)	Tests
HeuristicConfig learned mode	run.py	test_heuristic_config.py
BoxCache + synapse-seeded selection	dataset_builder.py	test_dataset_builder.py
End-to-end training CLI	scripts/train.py	test_train_helpers.py

Outstanding empirical work (not architectural):

1. Establish a real-data val F1 baseline from the current training run.
2. Fetch more boxes (target: 100+) and train for more epochs once baseline is confirmed.
3. Enable GAT training once grammar F1 is stable.
4. Add a held-out test set (currently validation and training boxes are from the same pool).
5. Evaluate per-neuron F1 distribution in addition to mean F1.

1.9 7. Methods Summary

This section summarises the core algorithms in explicit form.

1.9.1 7.1 Scaffold Union-Find

Input: Agent-synapse hit matrix `synapse_hits[agent_i, syn_j]`, synapse table with `pre_pt_supervoxel_id`, `post_pt_supervoxel_id`.

Algorithm: 1. For each agent, collect the set of supervoxel IDs at visited synapses (pre and post). 2. Initialize union-find with each agent as its own set. 3. For each pair of agents sharing at least one supervoxel on the same polarity side, union their sets. 4. Output: partition of agents into scaffold groups.

Complexity: $O(N \times M)$ for N agents, M synapses per agent; in practice $\sim 10\times$ reduction in group count.

1.9.2 7.2 Merge Scoring and Beam Search

Input: Scaffold groups, path arrays, grammar model.

Algorithm: 1. Encode each scaffold group’s representative path(s) with the Transformer PathEncoder. 2. For each candidate merge pair (within `MERGE_RADIUS`, above `MERGE_OVERLAP_THRESHOLD`), compute $s = \text{MergeScorer}(\text{emb}_i, \text{emb}_j)$. 3. Run beam search: repeatedly merge the highest-scoring compatible pair until no pair exceeds threshold or beam is full. 4. Output: merged neuron clusters.

1.9.3 7.3 Line-Graph F1 Definition

True edges: For each synapse pair (i, j) , add edge iff they share the same `pre_root_id` or the same `post_root_id`.

Predicted edges: For each synapse pair (i, j) , add edge iff both are assigned to the same pre-neuron and/or same post-neuron in the `ConnectivityGraph`.

F1 = $2 \cdot \text{TP} / (2 \cdot \text{TP} + \text{FP} + \text{FN})$ over these edge sets.

1.9.4 7.4 Design Choices and Rationale

Choice	Rationale
Line-graph F1 as primary scalar	Closest box-scale proxy for downstream connectome correctness; directly measurable from CAVE root IDs.
Coordinate-free path descriptors	Reusable across volumes; avoids registration; geometry captured by edge_len, radius, curvature.
CAVE scaffold init	Exploits existing annotations; collapses search space before learned decisions.
Transformer + [CLS] token	Global fragment embedding; multi-modal fusion without hand-designed pooling.
Soft-F1 surrogate for GAT	Differentiable approximation of terminal metric; gradients align with F1.
HeuristicConfig.learned()	Spatial thresholds generate candidates; learned models make final accept/reject.

1.10 8. Related Work and Positioning

We position Neuronauts explicitly relative to the connectomics ecosystem.

1.10.1 8.1 Relationship to Infrastructure and Data

Tool / System	Role	Neuronauts Relationship
CAVE	Annotation versioning, synapse tables, root IDs, materialization	Consumes. We treat CAVE root IDs and synapse tables as inputs. We do not replace CAVE; we refine connectivity inferred from it.
CloudVolume / Neuroglancer	EM imagery, segmentation meshes, viewing	Consumes. We fetch EM chunks via CloudVolume. Neuroglancer is for human inspection; we are inference-only.
MICrONS / Minnie65	Ground-truth dataset	Trains and evaluates on. Synapse line-graph F1 is computed against CAVE root IDs as the working ground truth.

1.10.2 8.2 Relationship to Segmentation and Tracing

Tool / Approach	What it optimises	Neuronauts relationship
Flood-filling networks (FFN)	Pixel-wise segmentation accuracy, merge/split at voxel level	Complementary. FFN produces the segmentation that CAVE versions. We operate downstream: given segments and synapses, we refine which synapses belong to which neuron. We do not re-segment.

Tool / Approach	What it optimises	Neuronauts relationship
RoboEM	Trace completion, neurite path accuracy	Different objective. RoboEM optimises trace correctness; we optimise connectome correctness (synapse line-graph F1). Our paths are evidence for merge decisions, not the end product. Different target. NEURD uses morphology as the primary signal. We treat morphology (path geometry) as one input; the primary target is connectivity topology. Optional input. We use membrane fields to guide agents; we can use a 2.5D U-Net or fall back to Sobel. Membrane quality is not our terminal metric.
NEURD	Morphology-based proofreading	
2.5D / 3D U-Nets	Membrane prediction	

1.10.3 8.3 Relationship to Proofreading and Edit Imitation

Tool / Approach	What it does	Neuronauts relationship
Auto-proof, edit imitation	Learn from proofreader merge/split decisions	Aligned but extended. We could consume such labels as additional merge supervision. Our main addition is global root-consistency supervision, scaffold init, and end-to-end F1 training. Similar structure, different features and target. Our nodes are scaffold groups; features are learned path embeddings; training target is synapse line-graph F1, not segmentation metric.
Graph-based agglomeration	Merge segments using hand-crafted or learned affinity	

1.10.4 8.4 Detailed Comparison: NEURD, Guided Proofreading, and Neuronauts

Dimension	NEURD	Guided Proofreading (Edit Imitation)	Neuronauts v2
Input	Whole-neuron 3D meshes + synapses	Segmentation boundaries, large context around borders	EM volume chunks (~6 um) + CAVE synapse table

Dimension	NEURD	Guided Proofreading (Edit Imitation)	Neuronauts v2
Representation	Skeleton decomposed into non-branching segments; directed tree graph over mesh	Raw segmentation + CNN receptive field	Agent paths (450 steps) encoded as sequences; scaffold groups
Logic	Heuristic graph rules (graph filters) — interpretable, hand-crafted	Learned CNN trained on expert yes/no decisions	Learned Transformer + GAT; grammar scores, beam search, soft-F1
Training signal	None (rule-based); parameters tuned from proof-reader feedback	Imitation of expert merge/split decisions (local proxy)	Synapse line-graph F1 (terminal connectome metric)
Scope	Whole neuron, entire mesh	Local boundary regions	Box-scale; one 6 um cube at a time
Merge vs split	Merge errors only; strips erroneous sub-graphs (no extension)	Both; split classifier + inverted merge classifier	Merge decisions over scaffold groups; no explicit split correction

Dimension	NEURD	Guided Proofreading (Edit Imitation)	Neuronauts v2
Output	Cleaned mesh, annotated graph, synapses	Corrected segmentation boundaries	Connectivity graph (neurons, synapse ownership)
Evaluation	reasigned Synapse precision/recall vs manual proof-reading	Variation of information, correction speed	Line-graph F1 over synapse pairs
Design philosophy	Conservative high precision, accept false negatives	Speed up human workflow (7.5x) via suggestions	End-to-end: train directly on connectome correctness

NEURD vs Neuronauts. NEURD operates on complete neuron meshes, decomposes them into morphology-rich graphs (spine density, width jumps, branching angles), and applies heuristic rules to identify merge errors. It is morphology-first: features are computed from mesh geometry, and the goal is to produce a cleaned, morphology-annotated neuron suitable for downstream analysis. Neuronauts, by contrast, operates on small EM boxes with no mesh requirement. Our paths are lightweight traces through the volume; we use CAVE supervoxel IDs to collapse the search space before any learned decision. The terminal target is synapse line-graph F1 — a direct measure of connectome correctness — not synapse precision/recall over manually proofread neurons. NEURD’s strength is scale (whole MICrONS/H01 volumes) and interpretability (human-tunable rules); Neuronauts’ strength is end-to-end learning against the scientific objective.

Guided Proofreading (Edit Imitation) vs Neuronauts. Systems such as Guided Proofreading (Haehn et al., CVPR 2018) and related “auto-proof” or edit-imitation approaches train a CNN to imitate expert merge/split decisions at segmentation boundaries. The training target is “would a human accept this edit?” — a local proxy that can drift from connectome correctness when proofreaders are inconsistent or when the scientific goal (connectome accuracy) differs from the immediate task (acceptable merge). Neuronauts could consume such labels as additional merge supervision (same-root vs different-root pairs from proofread data), but our primary signal is global: CAVE root consistency and line-graph F1. We also use scaffold initialization to reduce the problem size before learning, and a GAT trained on the terminal metric to make globally consistent decisions. Edit imitation is complementary: we could add proofreader-accepted merges as positives and proofreader-rejected merges as negatives to our merge loss.

1.10.5 8.5 What Neuronauts Is Not

- **Not a replacement for CAVE or Neuroglancer.** It is a refinement layer over CAVE outputs.
- **Not a general-purpose segmenter.** It does not produce voxel-level segmentation.

- **Not a morphology-first system.** Morphology is evidence; connectivity correctness is the objective.
- **Not trained on trace accuracy alone.** Trace quality matters only as input to merge and graph assembly.

1.11 9. Conclusion

Neuronauts v2 implements a complete end-to-end connectome inference platform grounded in a single thesis: the correctness of the induced connectome — not local segmentation quality — is the right training target. The system starts from CAVE segmentations as a noisy scaffold (reducing search space $\sim 10\times$ for free), encodes neural path geometry in an isotropic coordinate-free representation shared across all tasks, and trains every component against synapse line-graph F1.

The architecture is complete. The next milestone is establishing and improving a real-data baseline F1, which unlocks the automated self-improvement loop (`scripts/codex_optimize.py`) for subsequent architecture and hyperparameter exploration.

1.12 References

1. MICrONS Consortium et al. Functional connectomics spanning multiple areas of mouse visual cortex. *Nature* 2021. <https://www.nature.com/articles/s41586-021-03778-x>
2. Li, P. H. et al. RoboEM: neurite reconstruction from 3D EM by AI-based direct image-to-trace translation. *Nature Methods* 2024. <https://www.nature.com/articles/s41592-024-02226-5>
3. Reimer, J. et al. NEURD: automated proofreading and feature extraction for connectomics. *Nature* 2025. <https://www.nature.com/articles/s41586-025-08660-5>
4. Haehn, D. et al. Guided proofreading of automatic segmentations for connectomics. *CVPR* 2018. https://openaccess.thecvf.com/content_cvpr_2018/html/Haehn_Guided_Proofreading_of_CVPR_2018_paper.html
5. Silversmith, W. cloud-volume. <https://github.com/seung-lab/cloud-volume>
6. CAVEconnectome. CAVEclient. <https://github.com/CAVEconnectome/CAVEclient>
7. Bae, J. A. et al. Digital museum of retinal ganglion cells with dense anatomy and physiology. *Cell* 2024. [https://www.cell.com/cell/fulltext/S0092-8674\(24\)00308-4](https://www.cell.com/cell/fulltext/S0092-8674(24)00308-4)
8. Veličković, P. et al. Graph Attention Networks. *ICLR* 2018. <https://arxiv.org/abs/1710.10903>
9. Vaswani, A. et al. Attention Is All You Need. *NeurIPS* 2017. <https://arxiv.org/abs/1706.03762>
10. Whitney, M. et al. CAVE: Connectome Annotation Versioning Engine. *Nature Methods* 2025. <https://www.nature.com/articles/s41592-024-02426-z>